RGSCenter

Frontend REST API и интеграция Next.js

Техническое руководство по контрактам API, cookie-аутентификации, CSRF, страницам /login и /devices, локальной разработке и развёртыванию.

> Статус документа
> Соответствует ветке codex/frontend-nextjs после коммитов `e108e8f`, `95c06b1` и `e172401`. Дата фиксации: 02.09.2026.

Компонент	Технология / версия
Backend	`Java, Spring Boot, Spring Security`
Frontend	`Next.js 16.3.4, React 19.2.8, TypeScript`
Контракт	`REST JSON, префикс /api/v1`
Сессия	`Server-side token + HttpOnly cookie`
Production	`Next.js static export + reverse proxy + Spring Boot`

Содержание

Раздел	Наименование
1	Назначение и границы первого инкремента
2	Архитектура frontend/backend
3	Общие правила REST API
4	Аутентификация и CSRF
5	Справочник endpoint авторизации
6	Endpoint статусов устройств
7	Реализация Next.js
8	Локальная разработка
9	Production-развёртывание
10	Ошибки и диагностика
11	Тестирование и критерии совместимости
12	Ограничения и дальнейшие инкременты
A	Приложение: примеры fetch/curl

> Ключевая идея
> Next.js не обращается к устройствам напрямую. Он читает стабильный JSON snapshot, который Spring Boot формирует из уже существующей витрины текущих данных. HTTP-запрос не запускает новый ping и не блокируется на COM/TCP-операции.

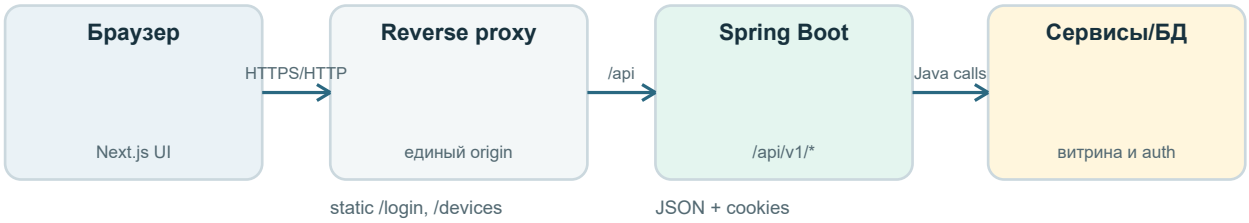
1. Назначение и границы первого инкремента

Цель инкремента - подготовить RGSCenter к постепенному переходу от Thymeleaf к отдельному frontend на Next.js без удаления существующих контроллеров и страниц. Реализован параллельный REST-контур, поэтому старый web-интерфейс продолжает работать.

- публичное получение CSRF metadata и cookie;
- JSON-вход, проверка текущего пользователя и выход;
- защищённый read-only snapshot статусов устройств;
- JSON 401/403 вместо HTML redirect для запросов /api/**;
- клиентские страницы /login и /devices;
- polling страницы устройств с периодом 10 секунд.

> Не входит в текущий инкремент
> REST-команды устройствам, управление сценарием измерения, административный UI, точные ACL по ролям, WebSocket/SSE и перенос всех Thymeleaf-страниц.

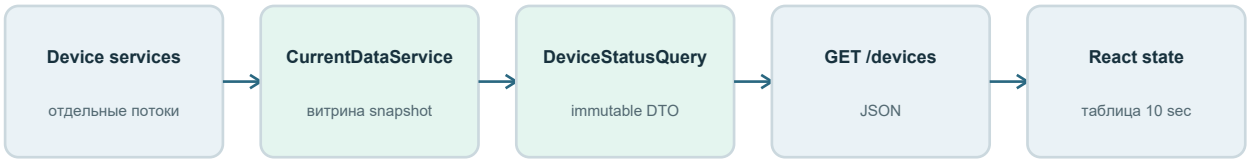
2. Архитектура frontend/backend



2.1. Ответственность компонентов

Компонент	Ответственность
Next.js	Форма входа, отображение статусов, периодические fetch-запросы, навигация при 401.
Reverse proxy	Единый origin: статические файлы frontend и проксирование /api в Spring Boot.
Spring Security	CSRF, восстановление principal из cookie, JSON 401/403, защита endpoint.
Auth services	Проверка BCrypt, выпуск/отзыв token session, хранение SHA-256 token hash.
DeviceStatusQueryService	Формирование immutable DTO из текущего состояния сервисов устройств.
Device services	Фоновый COM/LAN polling независимо от HTTP и frontend.

2.2. Поток данных устройства -> форма



Такое разделение защищает web-поток от таймаутов оборудования. В таблице показывается последнее зафиксированное состояние; поле generatedAt обозначает время сборки REST snapshot, a lastCheck - время последней проверки конкретного устройства.

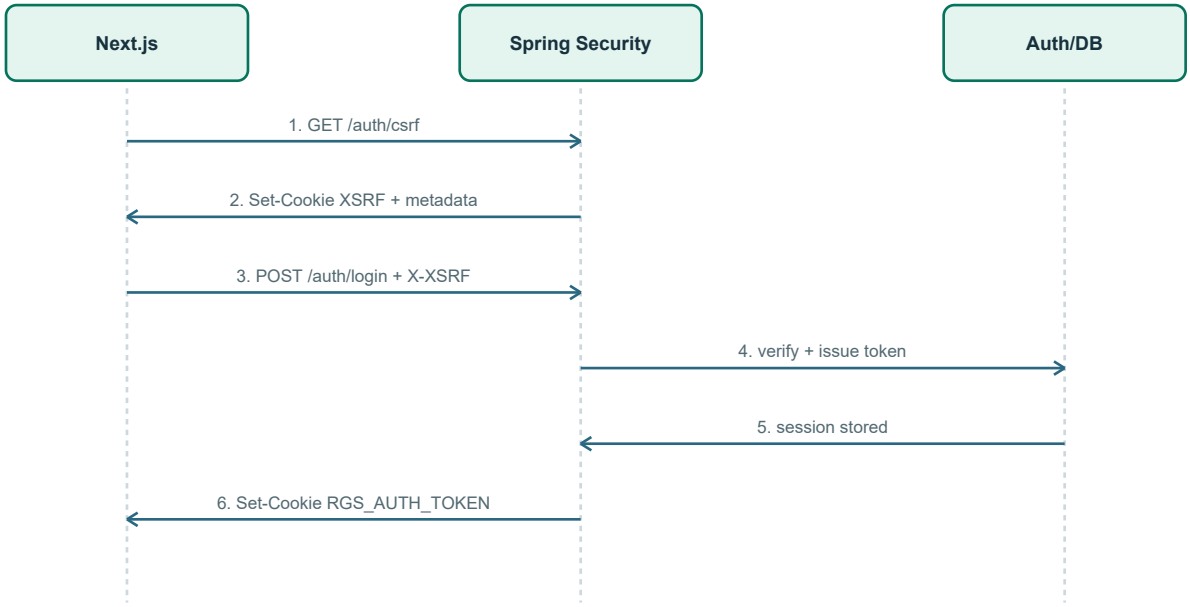
3. Общие правила REST API

Правило	Значение
Base path	`/api/v1`
Формат	`application/json; ошибки security - application/problem+json`
Время	`ISO-8601; Java Instant сериализуется с UTC offset Z`
Cookie	`fetch всегда использует credentials: include`
Cache	`auth-ответы no-store; устройства запрашиваются cache: no-store`
Версионирование	`Breaking change требует нового /api/v2 или совместимого расширения DTO`

> Совместимое расширение
> Добавление нового nullable-поля в JSON обычно совместимо. Переименование/удаление поля, изменение типа или семантики enum является breaking change.

4. Аутентификация и CSRF

RGSCenter использует stateless HTTP processing, но server-side token sessions. Spring HTTP session не создаётся. После проверки логина и BCrypt-хэша backend генерирует случайный 256-bit token. В браузер token передаётся через HttpOnly cookie, а в таблице auth_sessions хранится только SHA-256 hash.



4.1. Cookie

Cookie	Доступ из JS	Назначение	Атрибуты
XSRF-TOKEN	Да	Double-submit CSRF token	Path=/; SameSite=Strict; Secure по настройке
RGS_AUTH_TOKEN	Нет (HttpOnly)	Bearer token server-side session	Path=/; SameSite=Strict; Max-Age; Secure по настройке

4.2. Жизненный цикл

- Срок сессии по умолчанию: 8 часов (rgs.auth.session-duration).
- Активное подключение считается видимым в окне 90 секунд (rgs.auth.active-window).
- last_seen_at обновляется не чаще одного раза в 15 секунд, чтобы не писать в БД на каждый запрос.
- Logout устанавливает revoked_at и удаляет auth cookie; запись остаётся для audit timestamp.
- Истёкшие/отозванные записи старше 7 дней очищаются при последующем login.

> Production HTTPS
> При HTTPS обязательно установить RGSCENTER_COOKIE_SECURE=true. Иначе cookie не получит Secure. SameSite=Strict предполагает единый origin frontend и API.

5. Справочник endpoint авторизации

Метод	Path	Доступ	CSRF	Успех
GET	/api/v1/auth/csrf	public	нет	200 JSON + XSRF cookie
POST	/api/v1/auth/login	public	да	200 JSON + auth cookie
GET	/api/v1/auth/me	authenticated	нет	200 CurrentUser
POST	/api/v1/auth/logout	authenticated	да	204 + clear cookie

5.1. GET /api/v1/auth/csrf

Создаёт readable CSRF cookie и возвращает transport metadata без дублирования raw token в JSON.

```
HTTP 200
Set-Cookie: XSRF-TOKEN=<token>; Path=/; SameSite=Strict

{
  "cookieName": "XSRF-TOKEN",
  "headerName": "X-XSRF-TOKEN"
}
```

5.2. POST /api/v1/auth/login

```
POST /api/v1/auth/login
Content-Type: application/json
X-XSRF-TOKEN: <значение cookie XSRF-TOKEN>

{ "login": "operator", "password": "********" }

HTTP 200
Set-Cookie: RGS_AUTH_TOKEN=<opaque>; HttpOnly; Path=/; SameSite=Strict

{
  "authenticated": true,
  "expiresAt": "2026-09-02T16:00:00Z"
}
```

Перед новым входом существующий token из cookie отзывается. Неверные credentials возвращают 401 Problem Details. Login без корректного CSRF token возвращает 403.

5.3. GET /api/v1/auth/me

```
{
  "username": "operator",
  "role": "operator",
  "connected": 1,
  "multipleConnectionsWarning": false
}
```

Поле	Тип	Значение
`username`	`string`	Логин principal из текущей token session.
`role`	`string`	rolename из БД; регистр и naming пока не нормализованы API-контрактом.
`connected`	`integer`	Число активных, неотозванных и непросроченных сессий в active-window.
`multipleConnectionsWarning`	`boolean`	true, когда connected > 1.

5.4. POST /api/v1/auth/logout

Требует auth cookie, XSRF cookie и header. Успех - HTTP 204 без body; RGS_AUTH_TOKEN очищается через Max-Age=0. Повторное использование старого token после logout получает 401.

```
POST /api/v1/auth/logout
Cookie: RGS_AUTH_TOKEN=...; XSRF-TOKEN=...
X-XSRF-TOKEN: ...

HTTP 204 No Content
```

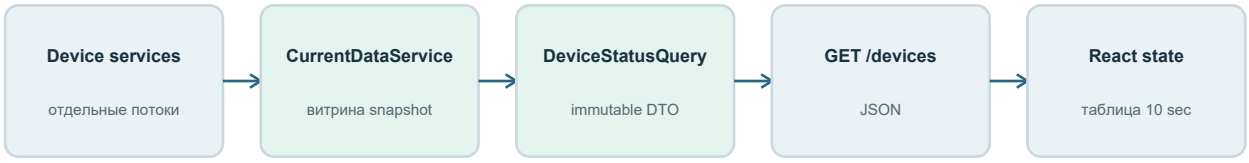
5.5. Стандартные ошибки security

```
{
  "title": "Authentication required",
  "status": 401,
  "detail": "Authentication is required to access this API resource"
}
```

HTTP	Когда	Frontend reaction
401	Нет, истёк или отозван auth token	Удалить локальное состояние и перейти на /login.
403	Нет/неверен CSRF либо недостаточно прав	Показать ошибку; при CSRF получить новый token и повторить осознанно.
404	Endpoint отсутствует в запущенной версии backend	Проверить версию JAR/source и перезапустить backend.

6. Endpoint статусов устройств

GET /api/v1/devices - защищённый read-only snapshot. CSRF не требуется, auth cookie обязательна.




```
GET /api/v1/devices
Cookie: RGS_AUTH_TOKEN=...
Accept: application/json
```

6.1. Верхний уровень DevicesResponse

Поле	Тип	Описание
`generatedAt`	`string/date-time`	Момент формирования snapshot.
`devices`	`DeviceResponse[]`	Сводные строки устройств; неизменяемая копия List.
`nicole`	`NicoleStatusResponse`	Связь, технический статус и последний PONG snapshot.
`shp`	`ShpStatusResponse`	Последние измерения SHP и вычисленный технический статус.

6.2. DeviceResponse

Поле	Тип	Допустимые значения / смысл
`code`	`string`	Стабильный machine-readable идентификатор.
`name`	`string`	Отображаемое имя.
`address`	`string`	IP, TCP endpoint или COM-порт.
`enabled`	`boolean`	Разрешено ли фоновое обслуживание конфигурацией.
`connectionStatus`	`enum`	ONLINE | OFFLINE | DISABLED
`lastCheck`	`string/date-time | null`	Последняя проверка/обмен или null.

> Важно
> connectionStatus показывает связь, а не техническую готовность. Для NICOLE и SHP технический статус вынесен в отдельные объекты nicole.technicalStatus и shp.technicalStatus.

6.3. NICOLE

Поле	Тип / значения	Описание
`connectionStatus`	`ONLINE | OFFLINE | DISABLED`	Результат корректного протокольного PING/PONG.
`technicalStatus`	`READY | ALERT | NO_DATA | DISABLED`	Готовность по последнему корректному PONG.
`warningText`	`string`	Декодированное описание warning flags.
`snapshot`	`object | null`	Полный набор значений PONG; null до первого валидного ответа.

Nicole snapshot содержит resetCause, ethernetUp, dhcpLeased, cryoControllerCommunication, cryocoolerStarted, advancedMode, уровни и давление азота, напряжения границ, частоту уровня, controllerTime, coolerErrorCode, warningFlags и receivedAt.

6.4. SHP

Поле	Тип	Описание
`ambientTemperatureCelsius`	`number | null`	Температура окружающей среды, °C.
`humidityPercent`	`number | null`	Относительная влажность, %.
`atmosphericPressureHpa`	`number | null`	Давление, hPa.
`leakageCurrent`	`number | null`	Ток утечки.
`chargeOutputVoltage`	`number | null`	Выходное напряжение charge circuit.
`technicalStatus`	`READY | ALERT | NO_DATA`	Результат проверки порогов.

6.5. Сокращённый пример ответа

```
{
  "generatedAt": "2026-09-02T08:15:30Z",
  "devices": [{
    "code": "camera", "name": "Camera",
    "address": "192.168.1.13", "enabled": true,
    "connectionStatus": "ONLINE",
    "lastCheck": "2026-09-02T08:15:27Z"
  }],
  "nicole": {
    "connectionStatus": "ONLINE", "technicalStatus": "READY",
    "warningText": "", "snapshot": { "cryocoolerStarted": true }
  },
  "shp": {
    "ambientTemperatureCelsius": 24.8, "humidityPercent": 40.9,
    "atmosphericPressureHpa": 719.1, "leakageCurrent": 11.4,
    "chargeOutputVoltage": 105.0, "technicalStatus": "READY"
  }
}
```

7. Реализация Next.js

7.1. Структура

```
frontend/
  next.config.ts
  package.json
  src/
    app/
      login/page.tsx
      login/login.module.css
      devices/page.tsx
      devices/devices.module.css
      layout.tsx
      globals.css
    lib/auth.ts
```

7.2. auth.ts

- csrfHeaders(): GET metadata, чтение XSRF-TOKEN из document.cookie, формирование динамического header.
- login(): POST credentials; 401 преобразуется в понятное сообщение пользователю.
- getCurrentUser(): возвращает CurrentUser либо null при 401.
- logout(): предварительно получает свежий CSRF token и выполняет защищённый POST.
- Все fetch используют credentials: include и cache: no-store.

7.3. /login

При открытии выполняется /auth/me. Действующая сессия перенаправляет на /devices. При submit форма получает CSRF, выполняет login, очищает password после ошибки и использует router.replace, чтобы защищённая страница не оставалась за login в history.

7.4. /devices

- проверяет пользователя через /auth/me;
- получает /api/v1/devices сразу после подтверждения сессии;
- повторяет snapshot каждые 10 000 ms;
- показывает сводку ONLINE/OFFLINE/DISABLED и локализует даты ru-RU;
- при 401 выполняет router.replace('/login');
- кнопка Выйти вызывает защищённый logout;
- multipleConnectionsWarning показывает число активных соединений.

> Текущая граница UI
> Страница /devices отображает devices[], но детальные блоки nicole и shp пока не выведены в Next.js. API уже предоставляет эти данные, поэтому это frontend-only следующий шаг.

8. Локальная разработка

Backend и frontend запускаются отдельно:

```
cd C:\Projects\RGSCenter\center
mvn clean spring-boot:run

cd C:\Projects\RGSCenter\frontend
npm install
npm run dev
```

URL	Назначение
http://localhost:8080	Spring Boot / существующий Thymeleaf
http://localhost:3000/login	Next.js login
http://localhost:3000/devices	Next.js devices
http://localhost:3000/api/v1/*	Dev rewrite -> http://localhost:8080/api/v1/*

next.config.ts включает rewrite только в development. Браузер работает с origin localhost:3000, а Next dev server проксирует /api в Spring Boot. Это сохраняет cookie как first-party и не требует CORS.

> Типичная причина HTTP 404/401 после добавления endpoint
> Backend был запущен до изменения исходников. Остановить старый Java-процесс, выполнить clean и перезапустить Spring Boot. GET /api/v1/auth/csrf должен вернуть HTTP 200.

9. Production-развёртывание

Production-конфигурация использует output: export и trailingSlash. npm run build формирует каталог frontend/out. Next.js server на NuvoPC не требуется: статические файлы обслуживает reverse proxy.

```
cd C:\Projects\RGSCenter\frontend
npm ci
npm run lint
npm run build

# Результат: frontend\out\
```

9.1. Маршрутизация reverse proxy

Path	Destination
/api/*	Spring Boot http://127.0.0.1:8080
/login/*, /devices/*, assets	Статический каталог frontend/out
остальные старые URL	Spring Boot или постепенно переносимый frontend

> Один origin - рекомендуемый вариант
> Раздельные домены/порты в production потребуют CORS и изменения cookie SameSite. Текущая модель безопаснее и проще при едином origin через Caddy или Nginx.

10. Ошибки и диагностика

Симптом	Вероятная причина	Проверка / действие
Login: HTTP 404	Старый backend без AuthRestController	Перезапустить backend; проверить GET /api/v1/auth/csrf.
Login: HTTP 403	Нет/не совпадает XSRF cookie/header	Сначала GET csrf; credentials: include; header X-XSRF-TOKEN.
Login: HTTP 401	Неверные credentials	Проверить пользователя/BCrypt hash; пароль не логировать.
Devices -> /login	Auth cookie истёк/отозван	Повторно войти; проверить RGS_AUTH_TOKEN в response, не читать его из JS.
Dev работает, export нет	Нет production reverse proxy для /api	Настроить proxy /api -> 8080; rewrite export не применяется.
Cookie не приходит по HTTPS	Неверный Secure/origin	RGSCENTER_COOKIE_SECURE=true и единый HTTPS origin.
Роль не локализована	API вернул lowercase rolename	Нормализовать role contract или roleLabel.toUpperCase().

Минимальная health-проверка auth API

```
curl.exe -i http://localhost:8080/api/v1/auth/csrf

# Ожидается:
# HTTP/1.1 200
# Set-Cookie: XSRF-TOKEN=...
# Content-Type: application/json
```

11. Тестирование и критерии совместимости

Интеграционные тесты фиксируют следующие обязательные свойства:

- полный lifecycle: csrf -> login -> me -> logout -> 401;
- login без CSRF = 403, неверный пароль с CSRF = JSON 401;
- logout без CSRF = 403, корректный logout очищает cookie;
- анонимный GET devices = application/problem+json 401 без HTML redirect;
- запрещённый POST devices = JSON 403;
- authenticated GET devices возвращает стабильные поля devices, nicole и shp.

```
cd C:\Projects\RGSCenter\center
mvn clean test

cd C:\Projects\RGSCenter\frontend
npm run lint
npm run build
```

На момент подготовки инкремента clean backend suite прошёл: 105 tests, 0 failures, 0 errors. Frontend lint и production build также завершились успешно; статические routes /login и /devices созданы.

12. Ограничения и дальнейшие инкременты

Приоритет	Доработка	Результат
P0	Точные ACL по ролям	ADMIN/ENGINEER/OPERATOR получают минимально необходимые endpoint.
P0	Production HTTPS + Secure cookie	Защита bearer cookie в сети.
P1	Вывод NICOLE/SHP в /devices	Использовать уже существующие поля ответа без backend change.
P1	OpenAPI 3 specification	Генерация TypeScript types и контрактная проверка CI.
P1	Единый API client	Typed fetch wrapper, timeout/AbortController, единая Problem Details обработка.
P1	Audit команд	Кто, когда, какой команде/устройству, результат и correlation ID.
P2	SSE/WebSocket	Push аварий READY -> ALERT без polling для critical events.
P2	Admin UI/password lifecycle	Пользователи, смена/сброс пароля, блокировки, удаление demo bootstrap.
P2	REST сценария измерения	start/stop/state/events с идемпотентностью и безопасными ACL.

12.1. Рекомендуемая последовательность

- Зафиксировать role enum и матрицу endpoint x role.
- Добавить OpenAPI для уже существующих пяти endpoint и генерировать frontend types.
- Вывести детальные NICOLE/SHP cards в Next.js.
- Переносить read-only страницы по одной, сохраняя Thymeleaf fallback.
- Только после стабилизации read API добавлять командные endpoint с audit, CSRF и ACL.
- Настроить Caddy/Nginx, HTTPS, Secure cookies и smoke tests на NuvoPC.

> Безопасность команд
> Endpoint управления физическими устройствами нельзя строить как простой CRUD. Нужны command ID, идемпотентность, очередь, timeout, состояние выполнения, audit и запрет повторной отправки после network retry.

Приложение A. Примеры клиента

A.1. TypeScript login

```
const csrf = await fetch("/api/v1/auth/csrf", {
  credentials: "include", cache: "no-store"
}).then(r => r.json());

const token = document.cookie
  .split(";")
  .map(v => v.trim())
  .find(v => v.startsWith(`${csrf.cookieName}=`))
  ?.split("=")[1];

await fetch("/api/v1/auth/login", {
  method: "POST", credentials: "include",
  headers: {
    "Content-Type": "application/json",
    [csrf.headerName]: decodeURIComponent(token ?? "")
  },
  body: JSON.stringify({ login, password })
});
```

A.2. Получение устройств

```
const response = await fetch("/api/v1/devices", {
  credentials: "include",
  cache: "no-store"
});

if (response.status === 401) {
  router.replace("/login");
  return;
}
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const snapshot = await response.json();
```

A.3. Logout

```
// Сначала получить свежий CSRF token/cookie
await fetch("/api/v1/auth/logout", {
  method: "POST",
  credentials: "include",
  cache: "no-store",
  headers: { "X-XSRF-TOKEN": xsrfToken }
});
```

A.4. Карта исходников

Назначение	Файл
Auth REST	`center/src/main/java/com/rgs/center/auth/AuthRestController.java`
Security	`center/src/main/java/com/rgs/center/auth/SecurityConfiguration.java`
JSON 401/403	`center/src/main/java/com/rgs/center/auth/ApiSecurityErrorHandler.java`
Devices REST	`center/src/main/java/com/rgs/center/api/devices/DeviceRestController.java`
Frontend auth	`frontend/src/lib/auth.ts`
Login page	`frontend/src/app/login/page.tsx`
Devices page	`frontend/src/app/devices/page.tsx`
Dev/export config	`frontend/next.config.ts`